



Writing basic MapReduce programs

This chapter covers

- Patent data as an example data set to process with Hadoop
- Skeleton of a MapReduce program
- Basic MapReduce programs to count statistics
- Hadoop's Streaming API for writing MapReduce programs using scripting languages
- Combiner to improve performance

The MapReduce programming model is unlike most programming models you may have learned. It'll take some time and practice to gain familiarity. To help develop your proficiency, we go through many example programs in the next couple chapters. These examples will illustrate various MapReduce programming techniques. By applying MapReduce in multiple ways you'll start to develop an intuition and a habit of "MapReduce thinking." The examples will cover simple tasks to advanced uses. In one of the advanced applications we introduce the Bloom filter, a data structure not normally taught in the standard computer science curriculum. You'll see that processing large data sets, whether you're using Hadoop or not, often requires a rethinking of the underlying algorithms.

We assume you already have a basic grasp of Hadoop. You can set up Hadoop, and you have compiled and run an example program, such as word counting from chapter 1. Let's use examples—from a real-world data set.

4.1 Getting the patent data set

To do anything meaningful with Hadoop we need data. Many of our examples will use patent data sets, both of which are available from the National Bureau of Economic Research (NBER) at <http://www.nber.org/patents/>. The data sets were originally compiled for the paper “The NBER Patent Citation Data File: Lessons, Insights and Methodological Tools.”¹ We use the citation data set `cite75_99.txt` and the patent description data set `apat63_99.txt`.

NOTE The data sets are approximately 250 MB each, which are small enough to make our examples runnable in Hadoop's standalone or pseudo-distributed mode. You can practice writing MapReduce programs using them even when you don't have access to a live cluster. The best part of Hadoop is that you can be fairly sure your MapReduce program will run on clusters of machines processing data sets 100 or 1,000 times larger with virtually no code changes.

A popular development tactic is to create a smaller, sampled subset of your large production data and call it the *development data set*. This development data set may only have several hundred megabytes. You develop your program in standalone or pseudo-distributed mode with the development data set. This gives your development process a fast turnaround time, the convenience of running on your own machine, and an isolated environment for debugging.

We have chosen these two data sets for our example programs because they're similar to most data types you'll encounter. First of all, the citation data encodes a graph in the same vein that web links and social networks are also graphs. Patents are published in chronological order; some of their properties resemble time series. Each patent is linked with a person (inventor) and a location (country of inventor). You can view them as personal or geographical data. Finally, you can look at the data as generic database relations with well-defined schemas, in a simple comma-separated format.²

4.1.1 The patent citation data

The patent citation data set contains citations from U.S. patents issued between 1975 and 1999. It has more than 16 million rows and the first few lines resemble the following:

¹ NBER Working Paper 8498, by Hall, B. H., A. B. Jaffe, and M. Tratjenberg (2001).

² There are more common data types than two data sets can possibly represent. An important one that's missing here is text, but you've already seen text used in the word count example. Other missing types include XML, image, and geolocation (the lat-long variety). Math matrix is not represented in general, although the citation graph can be interpreted as a sparse 0/1 matrix.

```
"CITING", "CITED"
3858241, 956203
3858241, 1324234
3858241, 3398406
3858241, 3557384
3858241, 3634889
3858242, 1515701
3858242, 3319261
3858242, 3668705
3858242, 3707004
...
```

The data set is in the standard comma-separated values (CSV) format, with the first line a description of the columns. Each of the other lines record one particular citation. For example, the second line shows that patent 3858241 cites patent 956203. The file is sorted by the citing patent. We can see that patent 3858241 cites five patents in total. Analyzing the data more quantitatively will give us deeper insights into it.

If you're only reading the data file, the citation data appears to be a bunch of numbers. You can "think" of this data in more interesting terms. One way is to visualize it as a graph. In figure 4.1 we've shown a portion of this citation graph. We can see that some patents are cited often whereas others aren't cited at all.³ Patents like 5936972 and 6009552 cite a similar set of patents (4354269, 4486882, 5598422) even though they don't cite each other. We use Hadoop to derive descriptive statistics about this patent data, as well as look for interesting patterns that aren't immediately obvious.

4.1.2 The patent description data

The other data set we use is the patent description data. It has the patent number, the patent application year, the patent grant year, the number of claims, and other meta-data about patents. Look at the first few lines of this data set. It's similar to a table in a relational database, but in CSV format. This data set has more than 2.9 million records. As in many real-world data sets, it has many missing values.

```
"PATENT", "GYEAR", "GDATE", "APPYEAR", "COUNTRY", "POSTATE", "ASSIGNEE",
➤ "ASSCODE", "CLAIMS", "NCLASS", "CAT", "SUBCAT", "CMADE", "CRECEIVE",
➤ "RATIOCIT", "GENERAL", "ORIGINAL", "FWDAPLAG", "BCKGTLAG", "SELFCTUB",
➤ "SELFCTLB", "SECDUPBD", "SECDLWBD"
3070801, 1963, 1096, "BE", "", 1, 269, 6, 69, 1, 0, , , , , , , ,
3070802, 1963, 1096, "US", "TX", 1, 2, 6, 63, 0, , , , , , , ,
3070803, 1963, 1096, "US", "IL", 1, 2, 6, 63, 9, 0.3704, , , , , , ,
3070804, 1963, 1096, "US", "OH", 1, 2, 6, 63, 3, 0.6667, , , , , , ,
3070805, 1963, 1096, "US", "CA", 1, 2, 6, 63, 1, 0, , , , , , ,
```

³ As with any data analysis, we must be careful when interpreting with limited data. When a patent doesn't seem to cite any other patents, it may be an older patent for which we have no citation information. On the other hand, more recent patents are cited less often because only newer patents can be aware of their existence.

```

3070806,1963,1096,, "US", "PA", ,1,,2,6,63,,0,,,,,,,,,
3070807,1963,1096,, "US", "OH", ,1,,623,3,39,,3,,0.4444,,,,,,
3070808,1963,1096,, "US", "IA", ,1,,623,3,39,,4,,0.375,,,,,,
3070809,1963,1096,, "US", "AZ", ,1,,4,6,65,,0,,,,,,,,,

```

The first row contains the name of a couple dozen attributes, which are meaningful only to patent specialists. Even without understanding all the attributes, it's still useful to have some idea of a few of them. Table 4.1 describes the first ten.

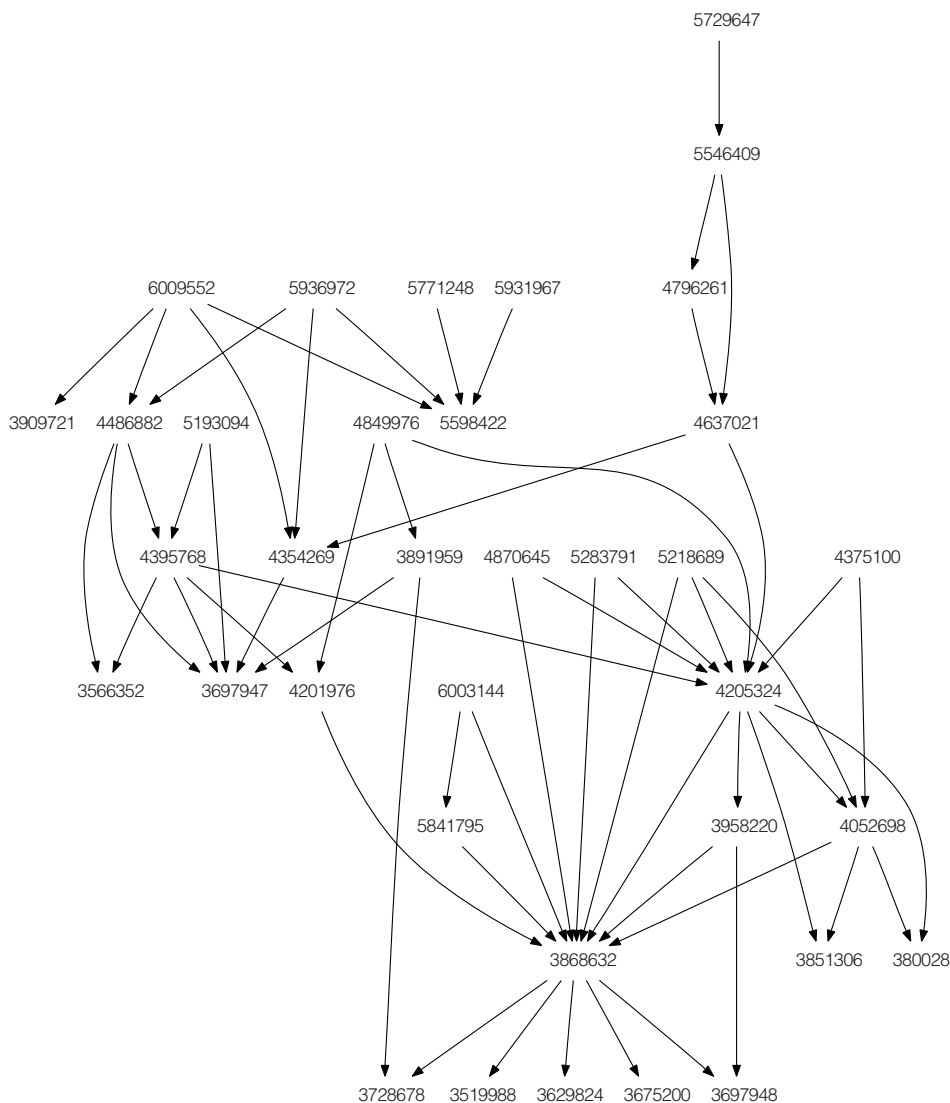


Figure 4.1 A partial view of the patent citation data set as a graph. Each patent is shown as a vertex (node), and each citation is a directed edge (arrow).

Table 4.1 Definition of the first 10 attributes in the patent description data set

Attribute name	Content
PATENT	Patent number
GYEAR	Grant year
GDATE	Grant date, given as the number of days elapsed since January 1, 1960
APPYEAR	Application year (available only for patents granted since 1967)
COUNTRY	Country of first inventor
POSTATE	State of first inventory (if country is U.S.)
ASSIGNEE	Numeric identifier for assignee (i.e., patent owner)
ASSCODE	One-digit (1-9) assignee type. (The assignee type includes U.S. individual, U.S. government, U.S. organization, non-U.S. individual, etc.)
CLAIMS	Number of claims (available only for patents granted since 1975)
NCLASS	3-digit main patent class

Now that we have two patent data sets, let's write Hadoop programs to process the data.

4.2 Constructing the basic template of a MapReduce program

We write most MapReduce programs in brief and as variations on a template. When writing a new MapReduce program, you generally take an existing MapReduce program and modify it until it does what you want. In this section, we write our first MapReduce program and explain its different parts. This program can serve as a template for future MapReduce programs.

Our first program will take the patent citation data and *invert* it. For each patent, we want to find and group the patents that cite it. Our output should be similar to the following:

```

1      3964859,4647229
10000  4539112
100000 5031388
1000006 4714284
1000007 4766693
1000011 5033339
1000017 3908629
1000026 4043055
1000033 4190903,4975983
1000043 4091523
1000044 4082383,4055371
1000045 4290571
1000046 5918892,5525001
1000049 5996916
```

```

1000051 4541310
1000054 4946631
1000065 4748968
1000067 5312208,4944640,5071294
1000070 4928425,5009029

```

We have discovered that patents 5312208, 4944640, and 5071294 cited patent 1000067. For this section we won't focus too much on the MapReduce data flow, which we've already covered in chapter 3. Instead we focus on the structure of a MapReduce program. We need only one file for the entire program as you can see in listing 4.1.

Listing 4.1 Template for a typical Hadoop program

```

public class MyJob extends Configured implements Tool {
    public static class MapClass extends MapReduceBase
        implements Mapper<Text, Text, Text, Text> {
        public void map(Text key, Text value,
            OutputCollector<Text, Text> output,
            Reporter reporter) throws IOException {
            output.collect(value, key);
        }
    }

    public static class Reduce extends MapReduceBase
        implements Reducer<Text, Text, Text, Text> {
        public void reduce(Text key, Iterator<Text> values,
            OutputCollector<Text, Text> output,
            Reporter reporter) throws IOException {

            String csv = "";
            while (values.hasNext()) {
                if (csv.length() > 0) csv += ",";
                csv += values.next().toString();
            }
            output.collect(key, new Text(csv));
        }
    }

    public int run(String[] args) throws Exception {
        Configuration conf = getConf();
        JobConf job = new JobConf(conf, MyJob.class);

        Path in = new Path(args[0]);
        Path out = new Path(args[1]);
        FileInputFormat.setInputPaths(job, in);
        FileOutputFormat.setOutputPath(job, out);

        job.setJobName("MyJob");
        job.setMapperClass(MapClass.class);
        job.setReducerClass(Reduce.class);

        job.setInputFormat(KeyValueTextInputFormat.class);
    }
}

```

```

        job.setOutputFormat(TextOutputFormat.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        job.set("key.value.separator.in.input.line", ",");

        JobClient.runJob(job);

        return 0;
    }

    public static void main(String[] args) throws Exception {
        int res = ToolRunner.run(new Configuration(), new MyJob(), args);

        System.exit(res);
    }
}

```

Our convention is that a single class, called `MyJob` in this case, completely defines each MapReduce job. Hadoop requires the `Mapper` and the `Reducer` to be their own static classes. These classes are quite small, and our template includes them as inner classes to the `MyJob` class. The advantage is that everything fits in one file, simplifying code management. But keep in mind that these inner classes are independent and don't interact much with the `MyJob` class. Various nodes with different JVMs clone and run the `Mapper` and the `Reducer` during job execution, whereas the rest of the job class is executed *only* at the client machine.

We investigate the `Mapper` and the `Reducer` classes in a while. Without those classes, the skeleton of the `MyJob` class is

```

public class MyJob extends Configured implements Tool {

    public int run(String[] args) throws Exception {
        Configuration conf = getConf();

        JobConf job = new JobConf(conf, MyJob.class);

        Path in = new Path(args[0]);
        Path out = new Path(args[1]);
        FileInputFormat.setInputPaths(job, in);
        FileOutputFormat.setOutputPath(job, out);

        job.setJobName("MyJob");
        job.setMapperClass(MapClass.class);
        job.setReducerClass(Reduce.class);

        job.setInputFormat(KeyValueTextInputFormat.class);
        job.setOutputFormat(TextOutputFormat.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        job.set("key.value.separator.in.input.line", ",");

        JobClient.runJob(job);

        return 0;
    }

    public static void main(String[] args) throws Exception {

```

```

        int res = ToolRunner.run(new Configuration(), new MyJob(), args);
        System.exit(res);
    }
}

```

The core of the skeleton is within the `run()` method, also known as the *driver*. The driver instantiates, configures, and passes a `JobConf` object named `job` to `JobClient.runJob()` to start the MapReduce job. (The `JobClient` class, in turn, will communicate with the `JobTracker` to start the job across the cluster.) The `JobConf` object will hold all configuration parameters necessary for the job to run. The driver needs to specify in `job` the input paths, the output paths, the `Mapper` class, and the `Reducer` class—the basic parameters for every job. In addition, each job can reset the default job properties, such as `InputFormat`, `OutputFormat`, and so on. One can also call the `set()` method on the `JobConf` object to set up any configuration parameter. Once you pass the `JobConf` object to `JobClient.runJob()`, it's treated as the master plan for the job. It becomes the blueprint for how the job will be run.

The `JobConf` object has many parameters, but we don't want to program the driver to set up all of them. The configuration files of the Hadoop installation are a good starting point. When starting a job from the command line, the user may also want to pass extra arguments to alter the job configuration. The driver can define its own set of commands and process the user arguments itself to enable the user to modify some of the configuration parameters. As this task is needed often, the Hadoop framework provides `ToolRunner`, `Tool`, and `Configured` to simplify it. When used together in the `MyJob` skeleton above, these classes enable our job to understand user-supplied options that are supported by `GenericOptionsParser`. For example, we have previously executed the `MyJob` class using this command line:

```
bin/hadoop jar playground/MyJob.jar MyJob input/cite75_99.txt output
```

Had we wanted to run the job only to see the mapper's output (which you may want to do for debugging purposes), we could set the number of reducers to zero with the option `-D mapred.reduce.tasks=0`.

```
bin/hadoop jar playground/MyJob.jar MyJob -D mapred.reduce.tasks=0
➡ input/cite75_99.txt output
```

It works even though our program doesn't explicitly understand the `-D` option. By using `ToolRunner`, `MyJob` will automatically support the options in table 4.2.

Table 4.2 Options supported by `GenericOptionsParser`

Option	Description
<code>-conf <configuration file></code>	Specify a configuration file.
<code>-D <property=value></code>	Set value for a <code>JobConf</code> property.
<code>-fs <local namenode:port></code>	Specify a <code>NameNode</code> , can be "local".

Table 4.2 Options supported by `GenericOptionsParser` (*continued*)

Option	Description
<code>-jt <local jobtracker:port></code>	Specify a JobTracker.
<code>-files <list of files></code>	Specify a comma-separated list of files to be used with the MapReduce job. These files are automatically distributed to all task nodes to be locally available.
<code>-libjars <list of jars></code>	Specify a comma-separated list of jar files to be included in the classpath of all task JVMs.
<code>-archives <list of archives></code>	Specify a comma-separated list of archives to be unarchived on all task nodes.

The convention for our template is to call the Mapper class `MapClass` and the Reducer class `Reduce`. The naming would seem more symmetrical if we call the Mapper class `Map`, but Java already has a class (interface) named `Map`. Both the Mapper and the Reducer extend `MapReduceBase`, which is a small class providing no-op implementations to the `configure()` and `close()` methods required by the two interfaces. We use the `configure()` and `close()` methods to set up and clean up the map (reduce) tasks. We won't need to override them except for more advanced jobs.

The signatures for the Mapper class and the Reducer class are

```
public static class MapClass extends MapReduceBase
    implements Mapper<K1, V1, K2, V2> {
    public void map(K1 key, V1 value,
                   OutputCollector<K2, V2> output,
                   Reporter reporter) throws IOException { }
}

public static class Reduce extends MapReduceBase
    implements Reducer<K2, V2, K3, V3> {
    public void reduce(K2 key, Iterator<V2> values,
                      OutputCollector<K3, V3> output,
                      Reporter reporter) throws IOException { }
}
```

The center of action for the Mapper class is the `map()` method and for the Reducer class the `reduce()` method. Each invocation of the `map()` method is given a key/value pair of types `K1` and `V1`, respectively. The key/value pairs generated by the mapper are outputted via the `collect()` method of the `OutputCollector` object. Somewhere in your `map()` method you need to call

```
output.collect((K2) k, (V2) v);
```

Each invocation of the `reduce()` method at the reducer is given a key of type `K2` and a list of values of type `V2`. Note that it must be the same `K2` and `V2` types used in the Mapper. The `reduce()` method will likely have a loop to go through all the values of type `V2`.

```

while (values.hasNext()) {
    V2? v = values.next();
    ...
}

```

The `reduce()` method is also given an `OutputCollector` to gather its key/value output, which is of type `K3/V3`. Somewhere in the `reduce()` method you'll call

```
output.collect((K3) k, (V3) v);
```

In addition to having consistent `K2` and `V2` types across `Mapper` and `Reducer`, you'll also need to ensure that the key and value types used in `Mapper` and `Reducer` are consistent with the input format, output key class, and output value class set in the driver. The use of `KeyValueTextInputFormat` means that `K1` and `V1` must both be type `Text`. The driver must call `setOutputKeyClass()` and `setOutputValueClass()` with the classes of `K2` and `V2`, respectively.

Finally, all the key and value types must be subtypes of `Writable`, which ensures a serialization interface for Hadoop to send the data around in a distributed cluster. In fact, the key types implement `WritableComparable`, a subinterface of `Writable`. The key types need to additionally support the `compareTo()` method, as keys are used for sorting in various places in the MapReduce framework.

4.3 *Counting things*

Much of what the layperson thinks of as statistics is counting, and many basic Hadoop jobs involve counting. We've already seen the word count example in chapter 1. For the patent citation data, we may want the number of citations a patent has received. This too is counting. The desired output would look like this:

```

1      2
10000  1
100000 1
1000006 1
1000007 1
1000011 1
1000017 1
1000026 1
1000033 2
1000043 1
1000044 2
1000045 1
1000046 2
1000049 1
1000051 1
1000054 1
1000065 1
1000067 3

```

In each record, a patent number is associated with the number of citations it has received. We can write a MapReduce program for this task. Like we said earlier, you hardly ever write a MapReduce program from scratch. You have an existing MapReduce

program that processes the data in a similar way. You copy that and modify it until it fits what you want.

We already have a program for getting the inverted citation index. We can modify that program to output the count instead of the list of citing patents. We need the modification only at the Reducer. If we choose to output the count as an `IntWritable`, we need to specify `IntWritable` in three places in the Reducer code. We called them V3 in our previous notation.

```
public static class Reduce extends MapReduceBase
    implements Reducer<Text, Text, Text, IntWritable> {
    public void reduce(Text key, Iterator<Text> values,
                      OutputCollector<Text, IntWritable> output,
                      Reporter reporter) throws IOException {

        int count = 0;
        while (values.hasNext()) {
            values.next();
            count++;
        }
        output.collect(key, new IntWritable(count));
    }
}
```

By changing a few lines and matching class types, we have a new MapReduce program. This program may seem a minor modification. Let's go through another example that requires more changes, but you'll see that the basic MapReduce program structure remains.

After running the previous example, we now have a data set that counts the number of citations for each patent. A neat exercise would be to count the counts. Let's build a histogram of the citation counts. We expect a large number of patents to have been only cited once, and a small number may have been cited hundreds of times. It would be interesting to see the distribution of the citation counts.

NOTE As the patent citation data set only covers patents issued between 1975 and 1999, the citation count is necessarily an underestimate. (Citations from patents outside of that period aren't counted.) We also don't deal with patents that supposedly have been cited zero times. Despite these caveats, the analysis will be useful.

The first step to writing a MapReduce program is to figure out the data flow. In this case, as a mapper reads a record, it ignores the patent number and outputs an intermediate key/value pair of `<citation_count, 1>`. The reducer will sum up the number of 1s for each citation count and output the total.

After figuring out the data flow, decide on the types for the key/value pairs—K1, V1, K2, V2, K3, and V3 for the input, intermediate, and output key/value pairs. Let's use the `KeyValueTextInputFormat`, which automatically breaks each input record into key/value pairs based on a separator character. The input format produces K1 and V1

as `Text`. We choose to use `IntWritable` for `K2`, `V2`, `K3`, and `V3` because we know those data must be integers and it's more efficient to use `IntWritable`.

Based on the data flow and the data types, you'll be able to see the final program shown in listing 4.2 and understand what it's doing. You can see that it's structurally similar to the other MapReduce programs we've seen so far. We go into details about the program after the listing.

Listing 4.2 CitationHistogram.java: count patents cited once, twice, and so on

```
public class CitationHistogram extends Configured implements Tool {

    public static class MapClass extends MapReduceBase
        implements Mapper<Text, Text, IntWritable, IntWritable> {

        private final static IntWritable uno = new IntWritable(1);
        private IntWritable citationCount = new IntWritable();

        public void map(Text key, Text value,
            OutputCollector<IntWritable, IntWritable> output,
            Reporter reporter) throws IOException {

            citationCount.set(Integer.parseInt(value.toString()));
            output.collect(citationCount, uno);
        }
    }

    public static class Reduce extends MapReduceBase
        implements Reducer<IntWritable, IntWritable, IntWritable, IntWritable>
    {

        public void reduce(IntWritable key, Iterator<IntWritable> values,
            OutputCollector<IntWritable, IntWritable> output,
            Reporter reporter) throws IOException {

            int count = 0;
            while (values.hasNext()) {
                count += values.next().get();
            }
            output.collect(key, new IntWritable(count));
        }
    }

    public int run(String[] args) throws Exception {
        Configuration conf = getConf();

        JobConf job = new JobConf(conf, CitationHistogram.class);

        Path in = new Path(args[0]);
        Path out = new Path(args[1]);
        FileInputFormat.setInputPaths(job, in);
        FileOutputFormat.setOutputPath(job, out);

        job.setJobName("CitationHistogram");
        job.setMapperClass(MapClass.class);
        job.setReducerClass(Reduce.class);

        job.setInputFormat(KeyValueTextInputFormat.class);
        job.setOutputFormat(TextOutputFormat.class);
```

```

        job.setOutputKeyClass(IntWritable.class);
        job.setOutputValueClass(IntWritable.class);

        JobClient.runJob(job);

        return 0;
    }

    public static void main(String[] args) throws Exception {
        int res = ToolRunner.run(new Configuration(),
                                new CitationHistogram(),
                                args);

        System.exit(res);
    }
}

```

The class name is now `CitationHistogram`; all references to `MyJob` were changed to reflect the new name. The `main()` method is almost always the same. The driver is mostly intact. The input format and output format are still `KeyValueTextInputFormat` and `TextOutputFormat`, respectively. The main change is that the output key class and the output value class are now `IntWritable`, to reflect the new type for `K2` and `V2`. We've also removed this line:

```
job.set("key.value.separator.in.input.line", ",");
```

It sets the separator character used by `KeyValueTextInputFormat` to break each input line into a key/value pair. Previously it was a comma for processing the original patent citation data. By not setting this property it defaults to the tab character, which is appropriate for the citation count data.

The data flow for this mapper is similar to that of the previous mappers, only here we've chosen to define and use a couple class variables—`citationCount` and `uno`.

```

public static class MapClass extends MapReduceBase
    implements Mapper<Text, Text, IntWritable, IntWritable> {

    private final static IntWritable uno = new IntWritable(1);
    private IntWritable citationCount = new IntWritable();

    public void map(Text key, Text value,
                    OutputCollector<IntWritable, IntWritable> output,
                    Reporter reporter) throws IOException {

        citationCount.set(Integer.parseInt(value.toString()));
        output.collect(citationCount, uno);
    }
}

```

The `map()` method has one extra line for setting `citationCount`, which is for type casting. The reason for defining `citationCount` and `uno` in the class rather than inside the method is purely one of efficiency. The `map()` method will be called as many times as there are records (in a split, for each JVM). Reducing the number of objects created inside the `map()` method can increase performance and reduce garbage collection. As we pass `citationCount` and `uno` to `output.collect()`,

we're relying on the `output.collect()` method's contract to not modify those two objects.⁴

The reducer sums up the values for each key. It seems inefficient because we know all values are 1s (uno, to be exact). Why do we need to sum the count? We've chosen this route because it will be easier for us later if we choose to add a *combiner* to enhance performance. Unlike `MapClass`, the call to `output.collect()` in `Reduce` instantiates a new `IntWritable` rather than reuse an existing one.

```
output.collect(key, new IntWritable(count));
```

We can improve performance by using an `IntWritable` class variable. But the number of times `reduce()` is called is much smaller in this particular program, probably no more than a thousand times (across all reducers). We don't have much need to optimize this particular code.

Running the MapReduce job on the citation count data will show the following result. As we suspect, a large number (900K+) of patents have only one citation, whereas some have hundreds of citations. The most popular patent has 779 citations.

```
1      921128
2      552246
3      380319
4      278438
5      210814
6      163149
7      127941
8      102155
9      82126
10     66634
...
411    1
605    1
613    1
631    1
633    1
654    1
658    1
678    1
716    1
779    1
```

As this histogram output is only several hundred lines long, we can put it into a spreadsheet and plot it. Figure 4.2 shows the number of patents at various citation frequencies. The plot is on a log-log scale. When a distribution shows as a line in a log-log plot, it's considered to be a *power law* distribution. The citation count histogram seems to fit the description, although its approximately parabolic curvature also suggests a *log-normal* distribution.

As you've seen in our examples so far, a MapReduce program is often not very big, and you can keep a certain structure across them to simplify development. Most of the work is in thinking through the data flow.

⁴ We see in section 5.1.3 that this reliance will forbid the `ChainMapper` from using pass-by-reference.

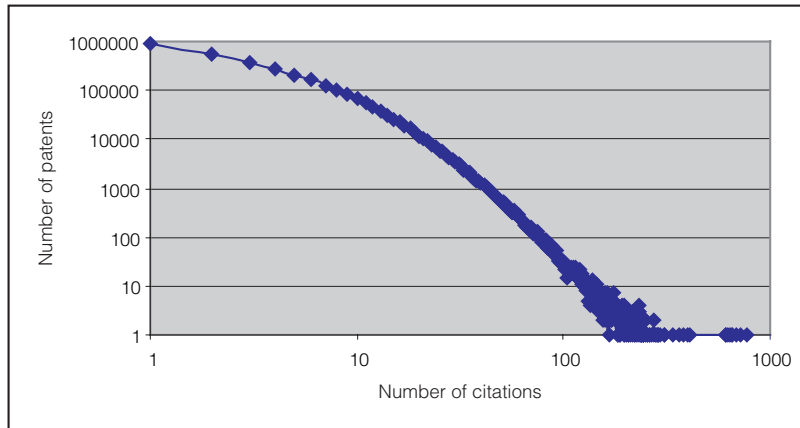


Figure 4.2 Plotting the number of patents at different citation frequencies. Many patents have one citation (or not at all, which is not shown on this graph). Some patents have hundreds of citations. On a log-log graph, this looks close enough to a straight line to be considered a power-law distribution.

4.4 Adapting for Hadoop's API changes

One of the main design goals driving toward Hadoop's major 1.0 release is a stable and extensible MapReduce API. As of this writing, version 0.20 is the latest release and is considered a bridge between the older API (that we use throughout this book) and this upcoming stable API. The 0.20 release supports the future API while maintaining backward-compatibility with the old one by marking it as deprecated. Future releases after 0.20 will stop supporting the older API. As of this writing, we don't recommend jumping into the new API yet for a couple reasons:

- 1 Many of Hadoop's own library classes in 0.20 aren't rewritten under the new API yet. You won't be able to use those classes if your MapReduce code uses the new API in 0.20.
- 2 Many still consider the most production-ready and stable version of Hadoop as of this writing to be 0.18.3. Some users are warming up to version 0.20, but we suggest you wait a little longer before going full production with it.⁵

By the time you read this the situation may be different. In this section we cover the changes the new API presents. Fortunately, almost all the changes affect only the basic MapReduce template. We rewrite the template under the new API to enable you to use it in the future.

⁵ You may be wondering about version 0.19. The general consensus is that its initial release was problematic and full of bugs. Some minor releases tried fixing the problems, but the community seems to want to skip straight to 0.20 instead.